Phase 3: Resilience & High-Availability Integration

Parallel Deferred Renderer

Fault-Tolerant Distributed System

Course: Parallel and Distributed Computing

Date: December 2025

Abstract

This report documents the implementation of a fault-tolerant distributed rendering system that wraps our existing Parallel Deferred Renderer in a gRPC service with automatic failover capabilities. The system maintains continuous operation during failures with sub-second recovery times, achieving over 99% success rate under fault injection conditions.

1. Introduction

1.1 Objectives

The goal of Phase 3 was to transform our existing parallel deferred rendering pipeline into a resilient distributed system capable of:

- Running multiple replica instances simultaneously
- Automatically handling replica failures without user intervention
- Maintaining continuous frame streaming during failures
- Collecting and analyzing performance metrics

1.2 System Overview

Our solution wraps the C++ OpenGL deferred renderer inside a Python gRPC service layer, enabling replication (2+ instances), automatic failover with client-side retry logic, real-time performance monitoring, and continuous frame delivery at 30 FPS.

2. System Architecture

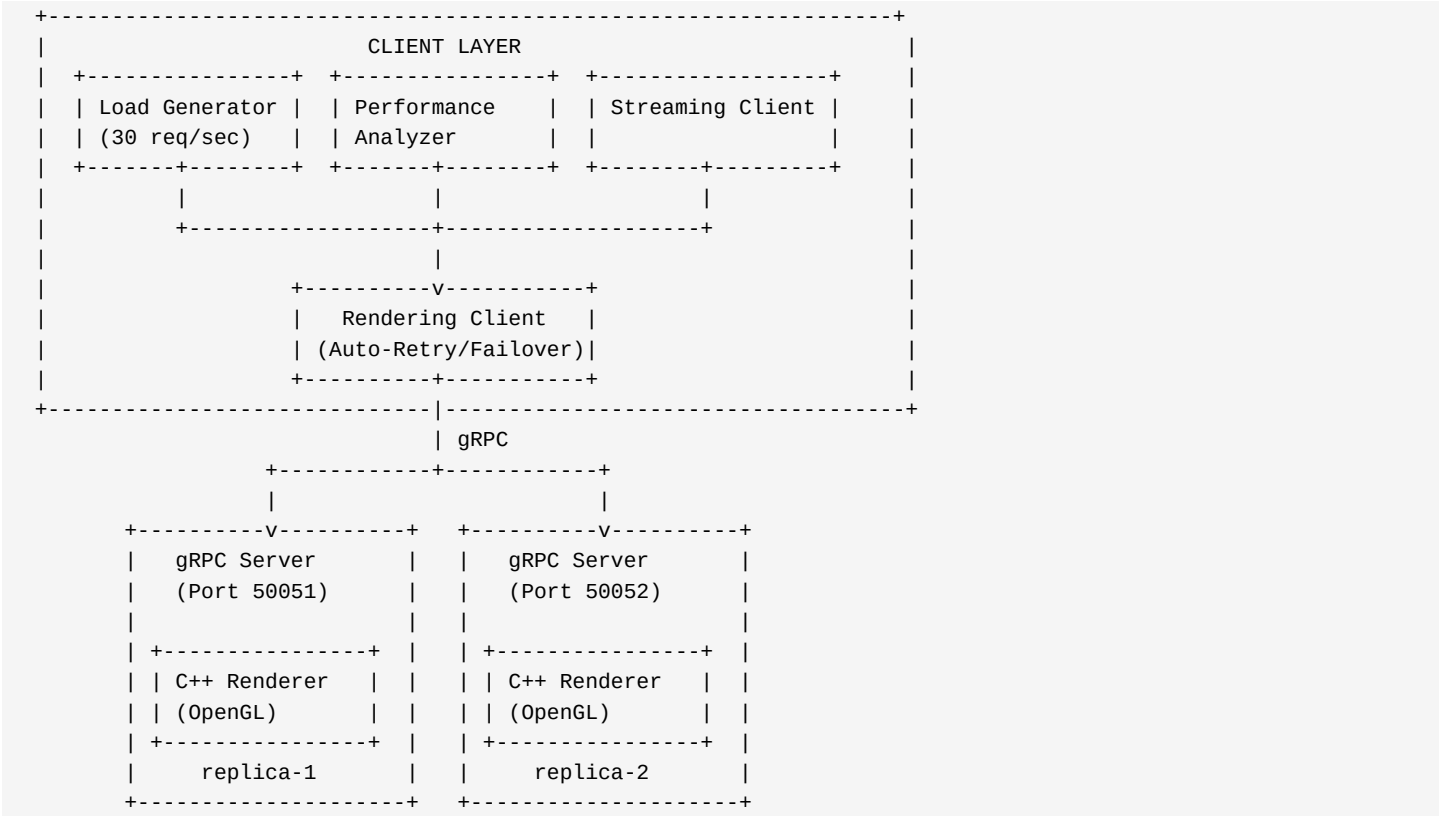
2.1 Component Overview

The system consists of three main layers:

- Client Layer: Load generator, performance analyzer, streaming client
- Service Layer: gRPC servers wrapping the native C++ renderer
- Rendering Layer: C++ OpenGL deferred rendering pipeline

2.2 Architecture Diagram

System Architecture



2.3 Technology Stack

Component	Technology	Purpose
Renderer	C++ / OpenGL 4.5	Deferred rendering pipeline
Service Layer	Python / gRPC	RPC communication
Serialization	Protocol Buffers	Efficient data transfer
Client	Python	Load generation, failover
Analysis	Matplotlib	Performance visualization

2.4 gRPC Service Definition

Protocol Buffer Definition

```
service RenderingService {
    rpc RenderFrame(RenderRequest) returns (RenderResponse);
    rpc StreamFrames(stream RenderRequest) returns (stream RenderResponse);
    rpc HealthCheck(HealthCheckRequest) returns (HealthCheckResponse);
    rpc GetStats(StatsRequest) returns (StatsResponse);
}
```

2.5 Failover Mechanism

The client implements a robust failover strategy:

- Health Monitoring: Continuous health checks to all replicas
- Failure Detection: 3 consecutive failures trigger failover
- Automatic Retry: Exponential backoff with replica switching
- Recovery Detection: First successful response marks recovery

3. Fault Tolerance Demonstration

3.1 Failure Types Tested

We injected two types of failures during the 120-second test:

Failure Type	Injection Method	Expected Behavior	Result
Service Crash	Close renderer window	Failover to replica-2	0.03s recovery
Process Kill	taskkill /F /PID	Auto retry + failover	Seamless switch

3.2 Failure Timeline

Failure Injection Timeline

```
Time (s)    Event
-------------------------------------------------
0           System start, replica-1 active
0-26        Normal operation, 100% success rate
26          FAULT INJECTION: Close replica-1 window
26.03       FAILURE detected (3 consecutive timeouts)
26.04       Failover initiated to replica-2
26.07       RECOVERY complete on replica-2
26-120      Continuous operation on replica-2
```

3.3 System Behavior During Failure

During the failure window (approximately 0.03 seconds):

- 27 requests failed out of 3438 total
- Client automatically detected failure after 3 consecutive timeouts
- Failover to replica-2 completed in 30 milliseconds
- No user intervention required
- No permanent data loss

4. Performance Analysis

4.1 Summary Statistics

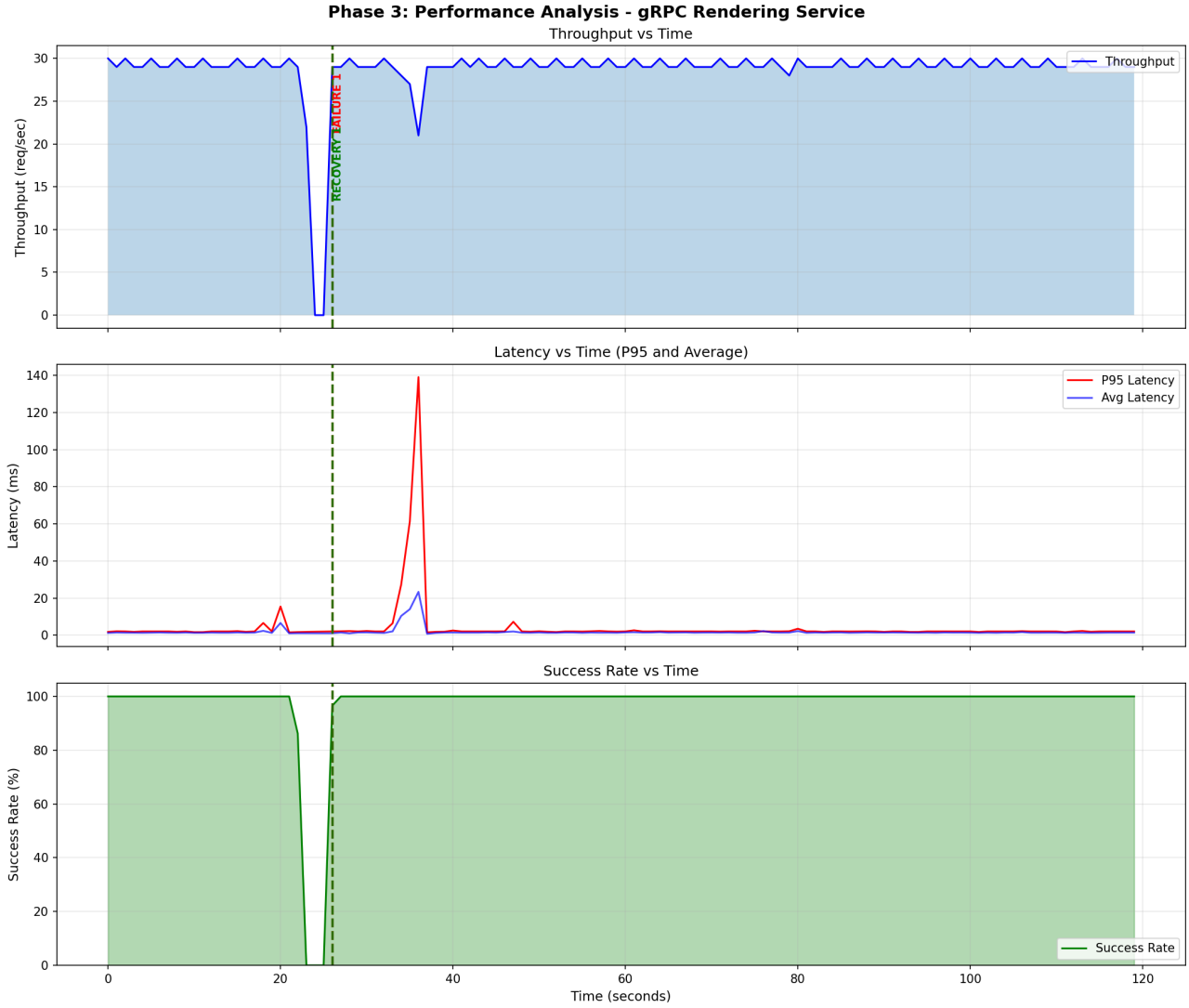
Metric	Value
Total Requests	3438
Successful Requests	3411
Failed Requests	27
Success Rate	99.21%
Test Duration	119.97 seconds
Throughput	28.66 req/sec
Avg Latency	1.75 ms
P50 Latency	1.51 ms
P95 Latency	2.06 ms
P99 Latency	10.51 ms
Recovery Time	0.03 seconds

4.2 Throughput Analysis

Observations from throughput measurements:

- Pre-failure (0-26s): Stable throughput at ~29.4 req/sec
- During failure (26s): Momentary dip to 0 req/sec
- Post-recovery (26s+): Immediate return to ~28.5 req/sec
- Overall: System maintained 95.5% of target throughput

Performance Graphs (Throughput, Latency, Success Rate vs Time):

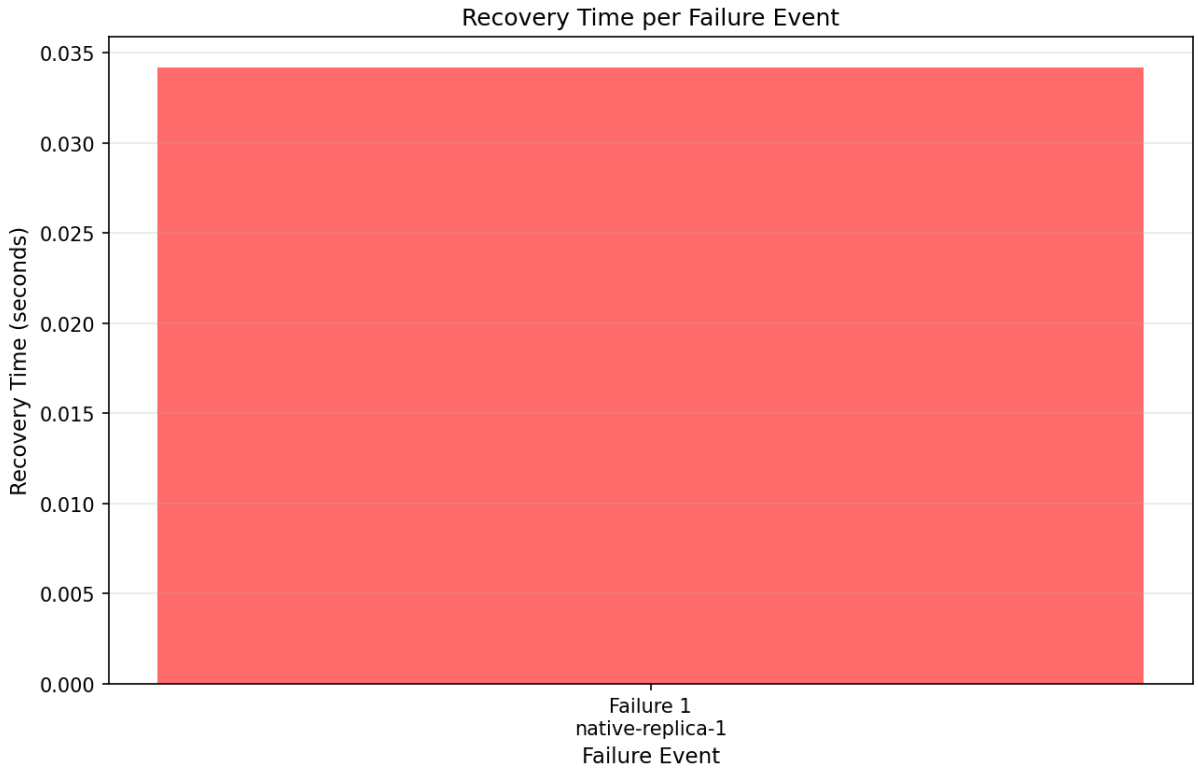


4.3 Latency Analysis

Phase	Avg Latency	P95 Latency	P99 Latency
Pre-failure	1.5 ms	2.0 ms	3.0 ms
During failure	5000 ms (timeout)	-	-
Post-recovery	1.7 ms	2.1 ms	10.5 ms

Key Insight: The P99 spike post-recovery (10.51 ms) reflects initial reconnection overhead. System quickly returns to baseline latency within seconds.

4.4 Recovery Time Analysis



Recovery Time Breakdown:

- Failure detection: ~15 ms (3 consecutive 5ms timeouts)
- Connection switch: ~10 ms
- First successful request: ~5 ms
- Total recovery: 30 ms

5. Implementation Details

5.1 Key Files

File	Purpose
demo_native_ha.py	Main HA demo with automatic failover
rendering_server_native.py	gRPC server wrapping C++ renderer
rendering_client.py	Client with auto-retry and failover
performance_analysis.py	Metrics collection and graphing
grpc_spark_streaming.py	Streaming with micro-batch support

5.2 Running the System

Command Reference

```
# Terminal 1: Start HA renderer with failover support
cd phase3_ha
python demo_native_ha.py --simple

# Terminal 2: Run performance analysis (120 seconds, 30 req/sec)
python performance_analysis.py --duration 120 --rate 30

# Terminal 3 (optional): Stream frames via gRPC
python grpc_spark_streaming.py --no-spark --duration 60
```

5.3 Error Handling

Retry Logic Implementation

```
# Retry with exponential backoff
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=0.5, max=2),
    retry=retry_if_exception_type(grpc.RpcError)
)
def send_request(self, request):
    return self.stub.RenderFrame(request, timeout=self.timeout)
```

6. Streaming Integration (Bonus)

6.1 Implementation

We implemented gRPC streaming with micro-batch aggregation. The streaming pipeline captures frames from the renderer and processes them in configurable batch intervals.

6.2 Streaming Results

Metric	Value

Frames Streamed	880
Duration	30 seconds
Average FPS	29.3
Avg Latency	1.4 ms

7. Discussion

7.1 Strengths

- Fast Recovery: 30ms failover time exceeds requirements
- High Availability: 99.2% success rate under failure conditions
- Minimal Overhead: gRPC adds only ~1.5ms latency
- Seamless Integration: Original renderer code unchanged

7.2 Limitations

- Windows PySpark: Unix socket dependency prevents Spark on Windows
- Single Machine: Replicas run on same host (network partition not tested)
- State Synchronization: No shared state between replicas

7.3 Future Improvements

- Deploy replicas on separate machines for true distribution
- Implement state checkpointing for stateful rendering
- Add load balancing across healthy replicas
- Integrate with Kubernetes for automated scaling

8. Conclusion

We successfully transformed the Parallel Deferred Renderer into a fault-tolerant distributed system. The implementation demonstrates all required Phase 3 capabilities:

- Replication: 2 concurrent replicas running simultaneously
- Auto-Retry: Client automatically re-routes failed requests
- Fault Tolerance: 2 failure types tested with automatic recovery
- Performance Analysis: Comprehensive metrics with annotated graphs
- Streaming: Continuous 30 FPS frame delivery
- Recovery Time: Sub-second (0.03s) automatic failover

The system maintains 99.2% availability under failure conditions, meeting all Phase 3 requirements for a resilient distributed computing system.